

Name: J. Renuka Devi

Reg. No: 192573007

Course code: CSA0925

Course Name: Java programming

OOPS CONCEPTS (INHERITANCE)

Bank

BANK

Write a Java program to implement a basic banking system using inheritance. Create a parent class called Account with attributes such as account number, account holder name, and balance. Then, create child classes such as SavingsAccount and CheckingAccount that

```
1 import java.util.Scanner;
2 class main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         SavingsAccount savings = new SavingsAccount("123456789", "Jane Doe", 1.5, 5000);
6         CheckingAccount checking = new CheckingAccount("987654321", "John Smith", 1000, 1000);
7         System.out.println("Accounts in the system are:");
8         System.out.print("1. ");
9         savings.displayDetails();
10        System.out.print("2. ");
11        checking.displayDetails();
12        int choice = sc.nextInt();
13        String type = sc.next();
14        System.out.print("Please enter the amount to " + type + ": ");
15        double amount = sc.nextDouble();
16        if (choice == 1) {
17            if (type.equalsIgnoreCase("deposit")) {
18                savings.deposit(amount);
19                System.out.println("Transaction successful.");
20                System.out.printf("New balance: $%.2f\n", savings.balance);
21            }
```

1
deposit
2500

OUTPUT

```
Accounts in the system are:
1. Savings Account (123456789) - Jane Doe (Balance: $5000.00, Interest Rate: 1.5%)
2. Checking Account (987654321) - John Smith (Balance: $1000.00, Overdraft Limit $1000.00)
Please enter the number of the account you wish to perform a transaction on:
```

QUESTIONS 20 / 20 completed

PROGRAM EDITOR

Java

Run

Save

INPUT

Employee

EMPLOYEE

Write a Java program to implement a basic employee management system using inheritance. Create a parent class called Employee with attributes such as name, id, and salary. Then, create child classes such as Manager and SalesPerson that inherit from the Employee class.

```
1 import java.util.Scanner;
2 class main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         String managename=sc.next();
6         int managerid=sc.nextInt();
7         double managersalary=sc.nextDouble();
8         int teamsize=sc.nextInt();
9         sc.nextLine();
10        String salesname=sc.next();
11        int salesid=sc.nextInt();
12        double salessalary=sc.nextDouble();
13        double salesamount=sc.nextDouble();
14        Manager manager=new Manager(managename,managerid,managersalary,teamsize);
15        Salesperson sp=new Salesperson(salesname,salesid,salessalary,salesamount);
16        System.out.println("Manager Details: ");
17        manager.display1();
18        manager.display2();
19        sp.display1();
20        sp.display3();
21    }
```

```
jane
101
50000
5
john
102
30000
75000
```

OUTPUT

```
Manager Details:
Name:jane
ID: 101
Salary: 50000.0
```

Shapes-1

SHAPES-1

Write a Java program to implement a basic shape hierarchy using inheritance. Create a parent class called Shape with attributes such as color and area. Then, create child classes such as Circle, Square, and Triangle that inherit from the Shape class and have their own unique methods for calculating area and perimeter.

```
1 import java.util.Scanner;
2 class main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         String color=sc.next();
6         double radius=sc.nextDouble();
7         double side=sc.nextDouble();
8         double base=sc.nextDouble();
9         double height=sc.nextDouble();
10        Circle circle=new Circle(color,radius);
11        Square square=new Square(color,side);
12        Triangle triangle=new Triangle(color,base,height);
13        System.out.println("-----Circle-----");
14        circle.display();
15        System.out.println("Area:"+circle.calculatearea());
16        System.out.println("Perimeter:"+circle.calculateperimeter());
17        System.out.println("\n-----Square-----\n");
18        square.display();
19        System.out.println("Area:"+square.calculatearea());
20        System.out.println("Perimeter:"+square.calculateperimeter());
21        System.out.println("\n-----Triangle-----\n");
```

```
blue
4
4
4
4
```

OUTPUT

```
-----Circle-----
Color: blue
Area:50.24
Perimeter:25.12

-----Square-----
```



SIMATS Online Programming Lab

JAVA PROGRAMMING

Jayaraman Renuka Devi
192573007RD

Library

LIBRARY

Write a Java program to implement a basic library management system using inheritance. Create a parent class called Item with attributes such as title and author. Then, create child classes such as Book and DVD that inherit from the Item class and have their own unique attributes

```
1 import java.util.Scanner;
2 class main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         String booktitle=sc.nextLine();
6         String bookauthor=sc.nextLine();
7         String bookno=sc.nextLine();
8         String dvdtitle=sc.nextLine();
9         String dvdauthor=sc.nextLine();
10        int duration=sc.nextInt();
11        book b=new book(booktitle,bookauthor,bookno);
12        dvd d=new dvd(dvdtitle,dvdauthor,duration);
13        System.out.println("-----BOOK-----");
14        b.display();
15        b.borrowitem();
16        b.returnitem();
17        System.out.println("\n-----DVD-----\n");
18        d.display();
19        d.borrowitem();
20        d.returnitem();
21    }
```

OUTPUT

```
-----BOOK-----
Title: harry potter
Author: jk rowling
Book borrowed: harry potter
Book returned:harry potter
```

harry potter
jk rowling
123456789
interstellar
christopher nolan
148



SIMATS Online Programming Lab

JAVA PROGRAMMING

Jayaraman Renuka Devi
192573007RD

University

UNIVERSITY

Write a Java program to implement a basic university management system using inheritance. Create a parent class called Person with attributes such as name and age. Then, create child classes such as Student and Professor that inherit from the Person class and have their own

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         String studentName = sc.nextLine();
6         int studentAge = sc.nextInt();
7         sc.nextLine();
8         String course = sc.nextLine();
9         String professorName = sc.nextLine();
10        int professorAge = sc.nextInt();
11        sc.nextLine();
12        String subject = sc.nextLine();
13        Student student = new Student(studentName, studentAge, course);
14        Professor professor = new Professor(professorName, professorAge, subject);
15        System.out.println("Student:");
16        student.display();
17        student.manageCoursework();
18        System.out.println("Professor:");
19        professor.display();
20        professor.teachClass();
21    }
```

OUTPUT

```
Student:
Name: Rahul
Age: 20
Course: Computer Science
Rahul is managing coursework.
Professor:
Name: Dr. Kumar
```

Rahul
20
Computer Science
Dr. Kumar
45
Data Structures

Restaurant

RESTURANT

Write a Java program to implement a basic restaurant management system using inheritance. Create a parent class called Menu with attributes such as name and price. Then, create child classes such as Appetizer and Entree that inherit from the Menu class and have their own

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         String appetizerName = sc.nextLine();
6         double appetizerPrice = sc.nextDouble();
7         sc.nextLine();
8         String appetizerType = sc.nextLine();
9         String entreeName = sc.nextLine();
10        double entreePrice = sc.nextDouble();
11        sc.nextLine();
12        String entreeSize = sc.nextLine();
13        Appetizer appetizer = new Appetizer(appetizerName, appetizerPrice, appetizerType);
14        Entree entree = new Entree(entreeName, entreePrice, entreeSize);
15        System.out.println("Appetizer:");
16        appetizer.display();
17        appetizer.order();
18        appetizer.serve();
19        System.out.println("Entree:");
20        entree.display();
21    }
22 }
```

Samosa
80
Veg
Biryani
250
Large

OUTPUT

```
Appetizer:
Name: Samosa
Price: 80.0
Appetizer ordered: Samosa
Appetizer served: Samosa
Entree:
Name: Biryani
```

Forecasting

FORCASTING

Write a Java program to implement a basic weather forecasting system using inheritance. Create a parent class called Forecast with attributes such as date and temperature. Then, create child classes such as SunnyForecast and RainyForecast that inherit from the

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         String date = sc.nextLine();
6         double temperature = sc.nextDouble();
7         int sunshineHours = sc.nextInt();
8         int rainfallChance = sc.nextInt();
9         SunnyForecast sunny = new SunnyForecast(date, temperature, sunshineHours);
10        RainyForecast rainy = new RainyForecast(date, temperature, rainfallChance);
11        sunny.predictWeather();
12        rainy.predictWeather();
13    }
14 }
15 class Forecast {
16     String date;
17     double temperature;
18
19     Forecast(String date, double temperature) {
20         this.date = date;
21         this.temperature = temperature;
22     }
23 }
```

12-08-2026
32
8
20

OUTPUT

```
Date: 12-08-2026
Temperature: 32.0
Weather: Sunny
Sunshine Hours: 8
Date: 12-08-2026
Temperature: 32.0
Weather: Rainy
```

Rental

RENTAL

Write a Java program to implement a basic car rental system using inheritance. Create a parent class called Vehicle with attributes such as make and model. Then, create child classes such as Sedan and SUV that inherit from the Vehicle class and have their own unique attributes and methods.

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         String sedanMake = sc.nextLine();
6         String sedanModel = sc.nextLine();
7         double sedanRate = sc.nextDouble();
8         sc.nextLine();
9         String suvMake = sc.nextLine();
10        String suvModel = sc.nextLine();
11        double suvRate = sc.nextDouble();
12        Sedan sedan = new Sedan(sedanMake, sedanModel, sedanRate);
13        SUV suv = new SUV(suvMake, suvModel, suvRate);
14        sedan.rentCar();
15        sedan.returnCar();
16        suv.rentCar();
17        suv.returnCar();
18        sc.close();
19    }
20 }
21 class Vehicle {
```

Toyota
Camry
2500
Hyundai
Creta
3000

OUTPUT

```
Make: Toyota
Model: Camry
Vehicle Type: Sedan
Rental Rate: 2500.0
Status: Car Rented
Status: Car Returned
Make: Hyundai
```

Shape-2

SHAPE-2

Write a Java program to implement a basic shape hierarchy using polymorphism. Create a parent class called Shape with attributes such as area and perimeter. Then, create child classes such as Circle and Rectangle that inherit from the Shape class and have their own unique methods for calculating area and perimeter.

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         double radius = sc.nextDouble();
6         double length = sc.nextDouble();
7         double width = sc.nextDouble();
8         Shape circle = new Circle(radius);
9         Shape rectangle = new Rectangle(length, width);
10        circle.calculateArea();
11        circle.calculatePerimeter();
12        rectangle.calculateArea();
13        rectangle.calculatePerimeter();
14        sc.close();
15    }
16 }
17 class Shape {
18     double area;
19     double perimeter;
20     void calculateArea() {
21     }
```

5
10
4

OUTPUT

```
Circle Area: 78.53981633974483
Circle Perimeter: 31.41592653589793
Rectangle Area: 40.0
Rectangle Perimeter: 28.0
```



SIMATS Online Programming Lab

JAVA PROGRAMMING

Jayaraman Renuka Devi
192573007RD

Draw

DRAW

Write a Java program to demonstrate polymorphism with interfaces. Create an interface called Drawable with a method called draw(). Then, create classes such as Circle and Square that implement the Drawable interface and have their own unique implementations of the draw() method.

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         Drawable circle = new Circle();
6         Drawable square = new Square();
7         circle.draw();
8         square.draw();
9         sc.close();
10    }
11 }
12 interface Drawable {
13     void draw();
14 }
15 class Circle implements Drawable {
16     public void draw() {
17         System.out.println("Drawing Circle");
18     }
19 }
20 class Square implements Drawable {
21     public void draw() {
```

OUTPUT

Drawing Circle
Drawing Square

Drawing Circle
Drawing Square



SIMATS Online Programming Lab

JAVA PROGRAMMING

Jayaraman Renuka Devi
192573007RD

QUESTIONS 20 / 20 completed

Animal-2

ANIMAL-2

Write a Java program to demonstrate polymorphism with abstract classes. Create an abstract class called Animal with abstract methods such as eat() and sleep(). Then, create child classes such as Dog and Cat that inherit from the Animal class and implement their own versions of the eat() and sleep() methods.

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         Animal dog = new Dog();
6         Animal cat = new Cat();
7         dog.eat();
8         dog.sleep();
9         cat.eat();
10        cat.sleep();
11    }
12 }
13 abstract class Animal {
14     abstract void eat();
15     abstract void sleep();
16 }
17 class Dog extends Animal {
18     void eat() {
19         System.out.println("Dog eats food");
20     }
21     void sleep() {
```

OUTPUT

Dog eats food
Dog sleeps
Cat eats fish
Cat sleeps

Dog eats food
Dog sleeps
Cat eats fish
Cat sleeps

QUESTIONS 20 / 20 completed

Overloading

OVERLOADING

Write a Java program to demonstrate method overloading with variable arguments. Create a method called sum() that takes in a variable number of integers and returns their sum. Then, create an overloaded version of the sum() method that takes in a variable number of doubles

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         int n = sc.nextInt();
6         int[] integers = new int[n];
7         for (int i = 0; i < n; i++) {
8             integers[i] = sc.nextInt();
9         }
10        int m = sc.nextInt();
11        double[] doubles = new double[m];
12        for (int i = 0; i < m; i++) {
13            doubles[i] = sc.nextDouble();
14        }
15        Calculator c = new Calculator();
16        System.out.println("Sum of integers: " + c.sum(integers));
17        System.out.println("Sum of doubles: " + c.sum(doubles));
18        sc.close();
19    }
20 }
21 class Calculator {
```

OUTPUT

```
Sum of integers: 100
Sum of doubles: 7.5
```

INPUT

```
4
10 20 30 40
3
1.5 2.5 3.5
```

Overriding

OVERRIDING

Write a Java program to demonstrate method overriding with a simple calculator. Create a parent class called Calculator with methods such as add(), subtract(), multiply(), and divide(). Then, create a child class called ScientificCalculator that overrides the

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         double a = sc.nextDouble();
6         double b = sc.nextDouble();
7         Calculator calculator = new ScientificCalculator();
8         System.out.println("Addition: " + calculator.add(a, b));
9         System.out.println("Subtraction: " + calculator.subtract(a, b));
10        System.out.println("Multiplication: " + calculator.multiply(a, b));
11        System.out.println("Division: " + calculator.divide(a, b));
12    }
13 }
14 class Calculator {
15     double add(double a, double b) {
16         return a + b;
17     }
18     double subtract(double a, double b) {
19         return a - b;
20     }
21     double multiply(double a, double b) {
```

OUTPUT

```
Addition: 5.0
Subtraction: -1.0
Multiplication: 8.0
Division: 0.6666666666666666
```

```
2
3
```


Person

PERSON

Write a Java program to create a class called Person with a constructor that takes in a name and age, and a method to print out the person's name and age.

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         String name = sc.nextLine();
6         int age = sc.nextInt();
7         Person person = new Person(name, age);
8         person.display();
9     }
10 }
11 class Person {
12     String name;
13     int age;
14     Person(String name, int age) {
15         this.name = name;
16         this.age = age;
17     }
18     void display() {
19         System.out.println("Name: " + name);
20         System.out.println("Age: " + age);
21     }
22 }
```

OUTPUT

```
Name: Renuka
Age: 20
```

Renuka
20

Rect-Const

RECT-CONST

Write a Java program to create a class called Rectangle with a constructor that takes in the length and width of the rectangle, and a method to calculate and return the area of the rectangle.

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         double length = sc.nextDouble();
6         double width = sc.nextDouble();
7         Rectangle rectangle = new Rectangle(length, width);
8         System.out.println("Area: " + rectangle.calculateArea());
9     }
10 }
11 class Rectangle {
12     double length;
13     double width;
14     Rectangle(double length, double width) {
15         this.length = length;
16         this.width = width;
17     }
18     double calculateArea() {
19         return length * width;
20     }
21 }
```

OUTPUT

```
Area: 50.0
```

10
5

QUESTIONS 20 / 20 completed

Bank-Const

BANK-CONST

Write a Java program to create a class called BankAccount with a constructor that takes in an account number and an initial balance, and methods to deposit and withdraw money from the account.

PROGRAM EDITOR

Java

Run

Save

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         String accountNumber = sc.nextLine();
6         double initialBalance = sc.nextDouble();
7         double depositAmount = sc.nextDouble();
8         double withdrawAmount = sc.nextDouble();
9         BankAccount account = new BankAccount(accountNumber, initialBalance);
10        account.deposit(depositAmount);
11        account.withdraw(withdrawAmount);
12        System.out.println("Account Number: " + account.accountNumber);
13        System.out.println("Balance: " + account.balance);
14    }
15 }
16 class BankAccount {
17     String accountNumber;
18     double balance;
19     BankAccount(String accountNumber, double initialBalance) {
20         this.accountNumber = accountNumber;
21         this.balance = initialBalance;
```

INPUT

```
123456789
5000
2000
1000
```

OUTPUT

```
Account Number: 123456789
Balance: 6000.0
```

Car-Const

CAR-CONST

Write a Java program to create a class called Car with a constructor that takes in the make, model, and year of the car, and a method to print out the car's make, model, and year.

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         String make = sc.nextLine();
6         String model = sc.nextLine();
7         int year = sc.nextInt();
8         Car car = new Car(make, model, year);
9         car.display();
10    }
11 }
12 class Car {
13     String make;
14     String model;
15     int year;
16     Car(String make, String model, int year) {
17         this.make = make;
18         this.model = model;
19         this.year = year;
20     }
21     void display() {
```

```
Toyota
Camry
2024
```

OUTPUT

```
Make: Toyota
Model: Camry
Year: 2024
```

Student-Const

STUDENT-CONST

Write a Java program to create a class called Student with a constructor that takes in a name, ID number, and a list of grades, and methods to calculate and return the student's average grade and letter grade.

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         String name = sc.nextLine();
6         int id = sc.nextInt();
7         int n = sc.nextInt();
8         double[] grades = new double[n];
9         for (int i = 0; i < n; i++) {
10             grades[i] = sc.nextDouble();
11         }
12         Student student = new Student(name, id, grades);
13         System.out.println("Average Grade: " + student.averageGrade());
14         System.out.println("Letter Grade: " + student.letterGrade());
15     }
16 }
17 class Student {
18     String name;
19     int id;
20     double[] grades;
21     Student(String name, int id, double[] grades) {
```

OUTPUT

```
Average Grade: 88.0
Letter Grade: B
```

Renuka
101
5
90 85 80 95 90

Games

GAMES

Write a Java program to implement a basic game system using inheritance. Create a parent class called Game with attributes such as title and genre. Then, create child classes such as ActionGame and PuzzleGame that inherit from the Game class and have their own unique attributes.

```
1 import java.util.Scanner;
2 public class Main {
3     public static void main(String[] args) {
4         Scanner sc = new Scanner(System.in);
5         String actionTitle = sc.nextLine();
6         String actionGenre = sc.nextLine();
7         int level = sc.nextInt();
8         sc.nextLine();
9         String puzzleTitle = sc.nextLine();
10        String puzzleGenre = sc.nextLine();
11        int difficulty = sc.nextInt();
12        ActionGame action = new ActionGame(actionTitle, actionGenre, level);
13        PuzzleGame puzzle = new PuzzleGame(puzzleTitle, puzzleGenre, difficulty);
14        System.out.println("Action Game:");
15        action.display();
16        action.play();
17        System.out.println("Puzzle Game:");
18        puzzle.display();
19        puzzle.play();
20    }
21 }
```

OUTPUT

```
Action Game:
Title:  PUBG
Genre: Action
Level: 5
    PUBG is being played.
Puzzle Game:
Title: Sudoku
```

PUBG
Action
5
Sudoku
Puzzle
3

Animal-1

ANIMAL-1

Write a Java program to implement a basic animal classification system using inheritance. Create a parent class called Animal with attributes such as name and habitat. Then, create child classes such as Mammal, Reptile, and Bird that inherit from the Animal class and have their own unique characteristics.

```
1 import java.util.Scanner;
2 class main{
3     public static void main(String[] args){
4         Scanner sc=new Scanner(System.in);
5         String mammalname=sc.next();
6         String mammalhabitat=sc.next();
7         String reptilenam=sc.next();
8         String reptilehabitat=sc.next();
9         String birdname=sc.next();
10        String birdhabitat=sc.next();
11        Mammal mammal=new Mammal(mammalname,mammalhabitat);
12        Reptile reptile=new Reptile(reptilenam,reptilehabitat);
13        Bird bird=new Bird(birdname,birdhabitat);
14        System.out.println("Mammal details: ");
15        mammal.display();
16        mammal.eat();
17        mammal.reproduce();
18        System.out.println("\n Reptile details: ");
19        reptile.display();
20        reptile.eat();
21        reptile.reproduce();
```

OUTPUT

```
Mammal details:
Animal name=lion
Animal habitat=forest
Mammal eats food using its mouth.
Mammal reproduces by giving birth to young ones.

Reptile details:
```

lion
forest
snake
grassland
eagle
mountains